Monad logger

Hackage: https://hackage.haskell.org/package/monad-logger-0.3.37

```
-- Control.Monad.Logger
type LogLine = (Loc, LogSource, LogLevel, LogStr)
type LogSource = Text
data Loc = Loc {
  loc_filename :: String,
  loc_package :: String,
  loc_module :: String,
  loc_start :: CharPos,
  loc_end :: CharPos
}
data LogStr
class ToLogStr s where
  toLogStr :: s -> LogStr

data LogLevel = LevelDebug | LevelInfo | LevelWarn |
LevelError | LevelOther            -- LevelDebug <
LevelOther
instance Eq LogLevel, Ord LogLevel, ...


-- Logger
class Monad m => MonadLogger m where
 monadLoggerLog :: ToLogStr msg => Loc -> LogSource ->
LogLevel -> msg -> m ()

class (MonadLogger m, MonadIO m) => MonadLoggerIO m
where
 askLoggerIO :: m (Loc -> LogSource -> LogLevel ->
LogStr -> IO ())

fromLogStr :: LogStr ->
ByteString
        -- Data.ByteString i.e. strict
defaultLoc :: Loc
defaultLogStr :: Loc -> LogSource -> LogLevel -> LogStr
-> LogStr
defaultOutput :: Loc -> LogSource -> LogLevel -> LogStr
-> IO ()


-- LoggingT
newtype LoggingT m a = LoggingT {
 runLoggingT :: (Loc -> LogSource -> LogLevel -> LogStr
-> IO ()) -> m a
}
newtype NoLoggingT m a = NoLoggingT { runLoggingT :: m
a }

runStdoutLogging :: MonadIO m => LoggingT m a -> m a
runStderrLogging :: MonadIO m => LoggingT m a -> m a
```

```
runFileLogging ::  MonadBaseControl m => FilePath ->
LoggingT m a -> m a
filterLogger :: (LogSource -> LogLevel -> Bool) ->
LoggingT m a -> LoggingT m a
mapLoggingT :: (m a -> n b) -> LoggingT m a -> LoggingT
n b

newtype NoLoggingT m a = NoLoggingT { runNoLoggingT :: m
a }
mapNoLoggingT :: (m a -> n b) -> NoLoggingT m a ->
NoLoggingT n b

newtype WriterLoggingT m a = WriterLoggingT
{ unWriterLoggingT :: m (a, DList LogLine) }
execWriterLoggingT :: Functor m => WriterLoggingT m a ->
m [LogLine]
runWriterLoggingT :: Functor m => WriterLoggingT m a ->
m (a, [LogLine])

-- Log
liftLoc :: Loc -> Q Exp
logDebugN :: MonadLogger m => Text -> m ()
logDebugNS :: MonadLogger m => LogSource -> Text -> m ()
logDebugCS :: MonadLogger m => CallStack -> Text -> m ()
logDebug :: Q Exp
 -- MonadLogger m => Text -> m () with current location
logDebugS :: Q Exp
 - MonadLogger m => LogSource -> Text -> m () with
current location
logDebugSH :: Q Exp
 -- (MonadLogger m, Show s) => s -> m ()
...
```

Log Levels
LevelDebug <= LevelInfo <= LevelWarn <= LevelError <= LevelOther

e.g. basic setup

```
{-# LANGUAGE DerivingStrategies #-}
{-# LANGUAGE TemplateHaskell #-}
{-# LANGUAGE GeneralizedNewtypeDeriving #-}

import Control.Monad.Catch
import Control.Monad.Logger
import Control.Monad.State
import Data.Function
import Control.Functor

-- step: add logger to stack
newtype MyApp s a = MkMyApp (LoggingT (StateT s IO) a)
     -- step: include LoggingT
  deriving newtype
```

```
    ( Functor,
      Applicative,
      Monad,
      MonadIO,
      MonadThrow,
      MonadCatch,
      MonadState s,
      MonadLogger
      -- step: MonadLogger
    )

-- step: run
main :: IO ()
main = runMyApp doSomething LevelDebug ()

runMyApp :: MyApp s a -> LogLevel -> s -> IO a
     -- step: run
runMyApp (MkMyApp app) minLevel st =
  filterLogger (\_ lvl -> lvl >= minLevel) app
     -- filter by level
  & runStdoutLoggingT
      -- stdout logging
  & flip runStateT st
  <&> fst

-- e.g. use.
doSomething :: MyApp s ()
doSomething = do
  $(logDebug) "Hello"
  $(logError) "OOPS"
```

Prints

```
[DEBUG] Hello @(main:test.hs /Users/yuinishizawa/
Projects/test.hs:30:8)
[ERROR] OOPS @(main:test.hs /Users/yuinishizawa/
Projects/test.hs:31:8)
```

Usage: format logs

Use a custom formatter when calling **runLoggingT**

```
formatter :: Loc -> LogSource -> LogLevel -> LogStr ->
IO ()
formatter _ text lv bd = B.putStr $ fromLogStr $
  toLogStr ("LOG [" <> T.pack (show lv) <> "]" <> text)
<> bd <> "\n"

-- e.g. call
app :: LoggingT IO a
runLoggingT app formatter
```

Prints

```
LOG [LevelDebug] This is a test
LOG [LevelError] FATAL ERROR
```

Usage: log to multiple outputs e.g. to stdout + file
Use **defaultLogStr** + **fromLogStr** to compile log manually.

e.g. simple with IO

```
import Data.ByteString.Char8 qualified as B
import Control.Monad.Logger

run :: FilePath -> LoggingT IO a -> IO a
run logPath app = withFile logPath ReadWriteMode $
\handle -> runLoggingT app $
      ( \loc src level message -> do
          let message' = fromLogStr $ defaultLogStr loc
src level message
          when (level >= LevelInfo) $ B.putStr
message'                              -- log to stdout
          B.hPutStr handle message'
                               -- log to file
      )

-- e.g. call
app :: LoggingT IO a
run "info.log" app
```

e.g. general for any monad m

```
import Data.ByteString.Char8 qualified as B
import Control.Monad.Logger
import Control.Monad.Trans.Control

runAppLogger :: forall m a. MonadBaseControl IO m =>
FilePath -> LoggingT m a -> m a
runAppLogger logPath app = liftBaseOp (withFile logPath
WriteMode) runApp
 where
  runApp handle = runLoggingT app $ \loc src level
message -> do
    let message' = fromLogStr $ defaultLogStr loc src
level message
    when (level >= LevelInfo) $ liftBase $ B.putStr
message'
    liftBase $ B.hPutStr handle message'
```

Usage: **derivable** logger

Define a **base logger** which can be derived by via.

```
  newtype ViaLogger env m a = ...
      base logger

      env = environment for storing writer ByteString -> IO ()
  instance MonadLogger (ViaLogger env m) where ...

  data MyEnv = MyEnv { myLogger :: ByteString -> IO () }
  newtype MyApp a = MyApp (ReaderT MyEnv IO a)
      derive
    deriving MonadLogger via (ViaLogger MyEnv (ReaderT MyContext IO a))
```

e.g.
Define base logger

```
{-# LANGUAGE DerivingVia #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE GeneralizedNewtypeDeriving #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE TemplateHaskell #-}

import Control.Lens
import Control.Monad.Logger
import Control.Monad.Reader
import Data.ByteString (ByteString)
import Data.ByteString.Char8 qualified as B
import Data.Kind (Type)

type LogWriter = ByteString -> IO ()


class IsLoggerEnv env where
                                        -- environment for
storing writer
  _logWriter :: Lens' env LogWriter

newtype ViaLogger env m a = ViaLogger (m a)

  deriving newtype (Functor, Applicative, Monad,
MonadIO, MonadReader env)

instance (IsLoggerEnv env, MonadIO m, MonadReader env m)
=> MonadLogger (ViaLogger env m) where
  monadLoggerLog loc src level msg = do
    logger <- view _logWriter
    liftIO $ logger $ fromLogStr $ defaultLogStr loc src
level $ toLogStr msg
```

Derive

```
data MyEnv = MyEnv { myLogger :: LogWriter }

defaultMyEnv :: MyEnv
defaultMyEnv = MyEnv B.putStrLn
```

```
instance IsLoggerEnv MyEnv where
  _logWriter f env =
    f (myLogger env)
      <&> \x -> env{myLogger = x}

newtype MyApp a = MyApp (ReaderT MyEnv IO a)
  deriving newtype (Functor, Applicative, Monad,
MonadReader MyEnv, MonadIO)
  deriving (MonadLogger) via (ViaLogger MyEnv (ReaderT
MyEnv IO))                              -- derive via

runMyApp :: MyApp a -> IO a
runMyApp (MyApp app) = runReaderT app defaultMyEnv
```

usage

```
someApp :: MyApp
someApp = do
  $(logDebug) "ABC"
```